CSN342
Deep Learning

1. School offering the course	School Of Computing
2. Course Code	CSF441
3. Course Title	**Deep Learning**
4. Credits (L:T:P:C)	2:0:1:3
5. Contact Hours (L:T:P)	2:0:2
6. Prerequisites (if any)	-
7. Course Basket	-

Faculty & Course coordinator :
Dr. Anushikha Singh

SYLLABUS : Unit 2
Feed Forward Networks

- Multilayer Perceptron (MLP) and activation functions.
- Logistic Regression, Learning parameters, Loss functions,
- Empirical Risk Minimization.
- Gradient Descent, Backpropagation algorithm.
- Regularization techniques.
- Difficulty of training deep neural networks, Greedy layerwise training.

Types of Neural networks

- Feedforward Neural networks: CNN, Perceptron
- Feedback Neural networks: RNN

FEED FORWARD NETWORKS

- A feedforward neural network is one of the simplest types of artificial neural networks devised. In this network, the information moves in only one direction—forward—from the input nodes, through the hidden nodes (if any), and to the output nodes. There are no cycles or loops in the network.
- No memory of past inputs; each input is treated independently.

Examples:

- Perceptron, Multilayer perceptron
- Convolutional Neural Network

Architecture: Feed forward Neural Networks

The architecture of a feedforward neural network consists of three types of layers: the input layer, hidden layers, and the output layer. Each layer is made up of units known as neurons, and the layers are interconnected by weights.

- **Input Layer:** This layer consists of neurons that receive inputs and pass them on to the next layer. The number of neurons in the input layer is determined by the dimensions of the input data.
- **Hidden Layers:** These layers are not exposed to the input or output and can be considered as the computational engine of the neural network. Each hidden layer's neurons take the weighted sum of the outputs from the previous layer, apply an activation function, and pass the result to the next layer. The network can have zero or more hidden layers.
- **Output Layer:** The final layer that produces the output for the given inputs. The number of neurons in the output layer depends on the number of possible outputs the network is designed to produce.
- Each neuron in one layer is connected to every neuron in the next layer, making this a fully connected network. The strength of the connection between neurons is represented by weights, and learning in a neural network involves updating these weights based on the error of the output.

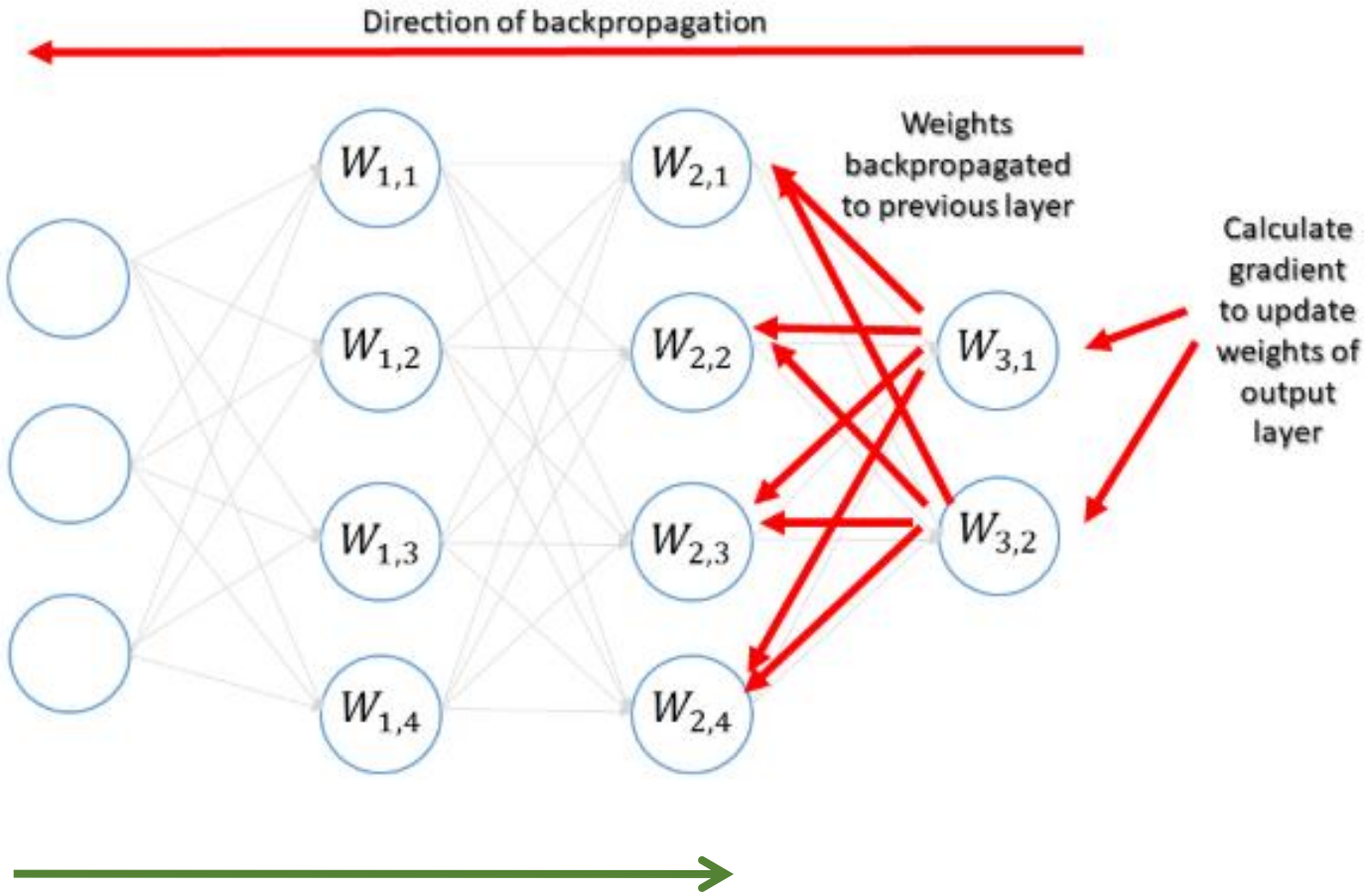
How Feedforward Neural Networks Work?

- The working of a feedforward neural network involves two phases: the feedforward phase and the backpropagation phase.
- **Feedforward Phase:** In this phase, the input data is fed into the network, and it propagates forward through the network. At each hidden layer, the weighted sum of the inputs is calculated and passed through an activation function, which introduces non-linearity into the model. This process continues until the output layer is reached, and a prediction is made.
- **Backpropagation Phase:** Once a prediction is made, the error (difference between the predicted output and the actual output) is calculated. This error is then propagated back through the network, and the weights are adjusted to minimize this error. The process of adjusting weights is typically done using a gradient descent optimization algorithm.

Training Feedforward Neural Networks

Training a feedforward neural network involves using a dataset to adjust the weights of the connections between neurons.

This is done through an iterative process where the dataset is passed through the network multiple times, and each time, the weights are updated to reduce the error in prediction. This process is known as gradient descent, and it continues until the network performs satisfactorily on the training data.

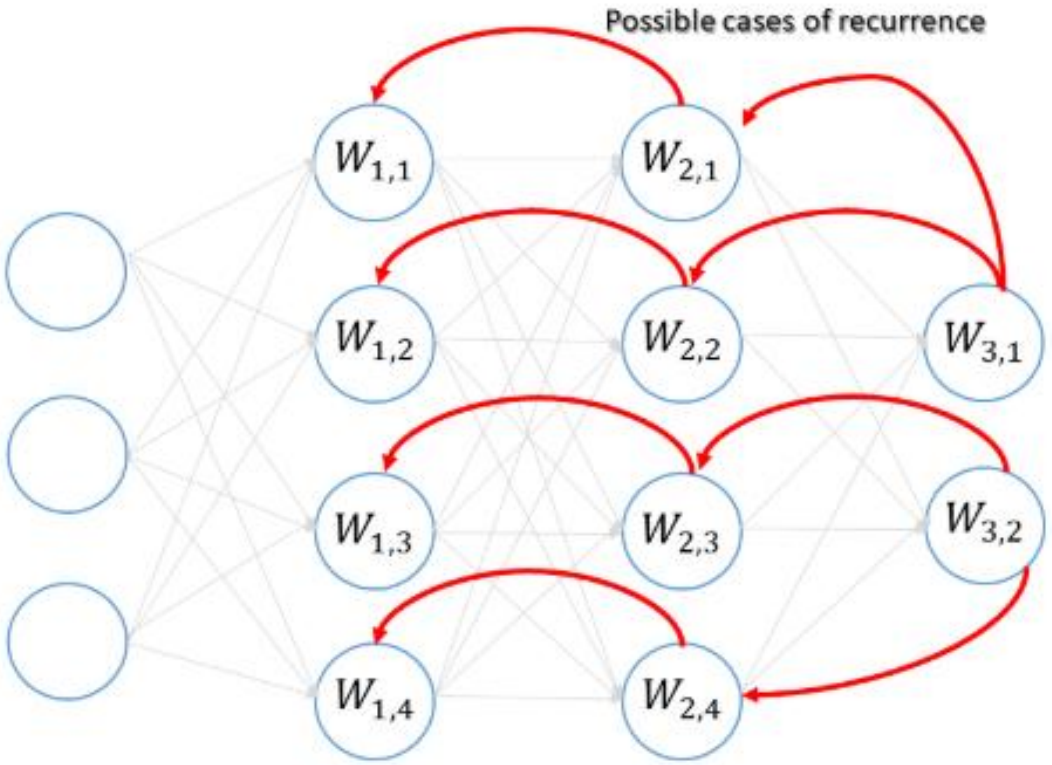

Direction of backpropagation
Weights backpropagated to previous layer
Calculate gradient to update weights of output layer
$W_{1,1}$
$W_{1,2}$
$W_{1,3}$
$W_{1,4}$
$W_{2,1}$
$W_{2,2}$
$W_{2,3}$
$W_{2,4}$
$W_{3,1}$
$W_{3,2}$
Forward Propagation

Feedback Neural Network

- A feed-back network, such as a **recurrent neural network** (**RNN**), features feed-back paths, which allow signals to use loops to travel in both directions. Neuronal connections can be made in any way. Since this kind of network contains loops, it transforms into a non-linear dynamic system that evolves during training continually until it achieves an equilibrium state.
- In research, RNN are the most prominent type of feed-back networks. They are an artificial neural network that forms connections between nodes into a directed or undirected graph along a temporal sequence. It can display temporal dynamic behavior as a result of this. RNNs may process input sequences of different lengths by using their internal state, which can represent a form of memory. They can therefore be used for applications like speech recognition or handwriting recognition.

How It Works

- **Information Flow:** In an RNN, the output from a neuron at time step ttt feeds back into the network as input for the next time step.
- **Memory Capability:** RNNs maintain a hidden state that evolves as they process each element of a sequence, capturing context and dependencies across time steps.
- **Weight Sharing:** The same weights are applied to all time steps, which helps in generalizing sequential patterns.


Possible cases of recurrence
$W_{1,1}$
$W_{2,1}$
$W_{1,2}$
$W_{2,2}$
$W_{3,1}$
$W_{1,3}$
$W_{2,3}$
$W_{3,2}$
$W_{1,4}$
$W_{2,4}$

Basic Components of Perceptron

A perceptron, the basic unit of a neural network, comprises essential components that collaborate in information processing.

- **Input Features:** The perceptron takes multiple input features, each input feature represents a characteristic or attribute of the input data.
- **Weights**: Each input feature is associated with a weight, determining the significance of each input feature in influencing the perceptron's output. During training, these weights are adjusted to learn the optimal values.
- **Summation Function**: The perceptron calculates the weighted sum of its inputs using the summation function. The summation function combines the inputs with their respective weights to produce a weighted sum.

- **Activation Function**: The weighted sum is then passed through an activation function. Perceptron uses **Heaviside step function** functions. **which take the summed values as input and compare with the threshold and provide the output as 0 or 1.**
- **Output**: The final output of the perceptron, is determined by the activation function's result. For example, in binary classification problems, the output might represent a predicted class (0 or 1).
- **Bias**: A bias term is often included in the perceptron model. The bias allows the model to make adjustments that are independent of the input. It is an additional parameter that is learned during training.
- **Learning Algorithm (Weight Update Rule)**: During training, the perceptron learns by adjusting its weights and bias based on a learning algorithm. A common approach is the perceptron learning algorithm, which **updates weights based on the difference between the predicted output and the true output.**

Types of Perceptron

- **Single-Layer Perceptron:** This type of perceptron is limited to learning linearly separable patterns. effective for tasks where the data can be divided into distinct categories through a straight line.
- **Multilayer Perceptron**: Multilayer perceptrons possess enhanced processing capabilities as they consist of two or more layers, adept at handling more complex patterns and relationships within the data.

Single layer Perceptron Vs Multilayer Perceptron

Feature	Perceptron	Multilayer Perceptron (MLP)
Structure	Single layer	Multiple layers (with hidden layers)
Problem Solving	Only linearly separable problems	Both linear and non-linear problems
Activation Function	Step function	Non-linear functions (ReLU, sigmoid, etc.)
Training Algorithm	Perceptron learning rule	Backpropagation with gradient descent
Use Cases	Simple, binary classification	Complex tasks like classification, regression, etc.

Activation function

- An activation function in the context of neural networks is a mathematical function applied to the output of a neuron. **The purpose of an activation function is to introduce non-linearity into the model, allowing the network to learn and represent complex patterns in the data.** Without non-linearity, a neural network would essentially behave like a linear regression model, regardless of the number of layers it has.
- The activation function decides whether a neuron should be activated or not by calculating the weighted sum and further adding bias to it.

Why do we need Non-linear activation function?

- A neural network without an activation function is essentially just a linear regression model. The activation function does the non-linear transformation to the input making it capable to learn and perform more complex tasks.

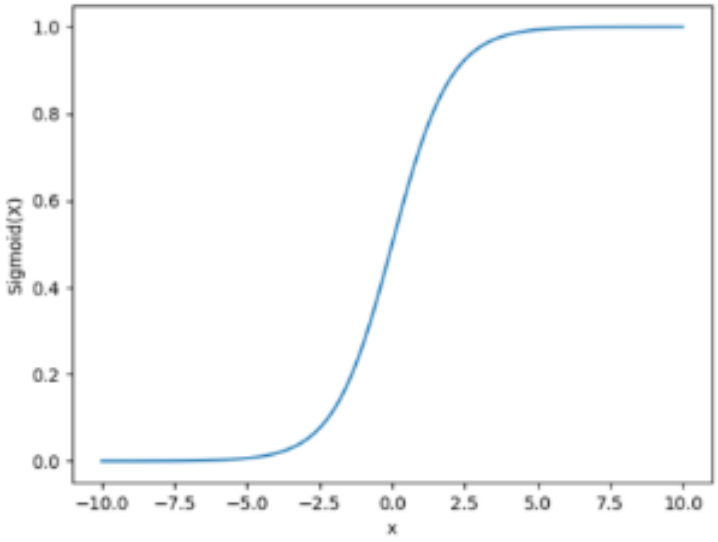
Variants of Activation Function

Linear Function

- **Equation :** Linear function has the equation similar to as of a straight line i.e. **y = x**
- No matter how many layers we have, if all are linear in nature, the final activation function of last layer is nothing but just a linear function of the input of first layer.
- **Range :** -inf to +inf
- **Uses : Linear activation function** is used at just one place i.e. output layer.
- **Issues :** If we will differentiate linear function to bring non-linearity, result will no more depend on *input "x"* and function will become constant, it won't introduce any ground-breaking behavior to our algorithm.

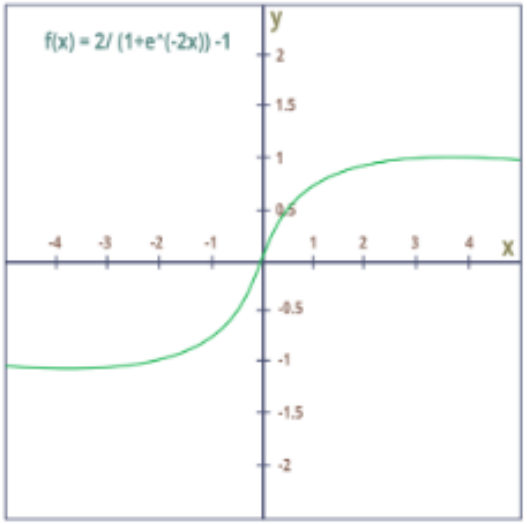
For example : Calculation of price of a house is a regression problem. House price may have any big/small value, so we can apply linear activation at output layer. Even in this case neural net must have any non-linear function at hidden layers.

Sigmoid Function



- It is a function which is plotted as **'S'** shaped graph.
- **Equation :** $A = 1/(1 + e^{-x})$
- **Nature :** Non-linear. Notice that X values lies between -2 to 2, Y values are very steep. This means, small changes in x would also bring about large changes in the value of Y.
- **Value Range :** 0 to 1
- **Uses :** Usually used in output layer of a binary classification, where result is either 0 or 1, as value for sigmoid function lies between 0 and 1 only so, result can be predicted easily to be ***1*** if value is greater than **0.5** and ***0*** otherwise.

Tanh Function



- The activation that works almost always better than sigmoid function is Tanh function also known as **Tangent Hyperbolic function**. It's actually mathematically shifted version of the sigmoid function. Both are similar and can be derived from each other.
- **Equation :-**
  f(x) = tanh(x) = 2/(1 + e-2x) – 1
  OR
  tanh(x) = 2 * sigmoid(2x) – 1
- **Value Range :-** -1 to +1
- **Nature :-** non-linear
- **Uses :-** Usually used in hidden layers of a neural network as it's values lies between **-1 to 1** hence the mean for the hidden layer comes out be 0 or very close to it, hence helps in *centering the data* by bringing mean close to 0. This makes learning for the next layer much easier.

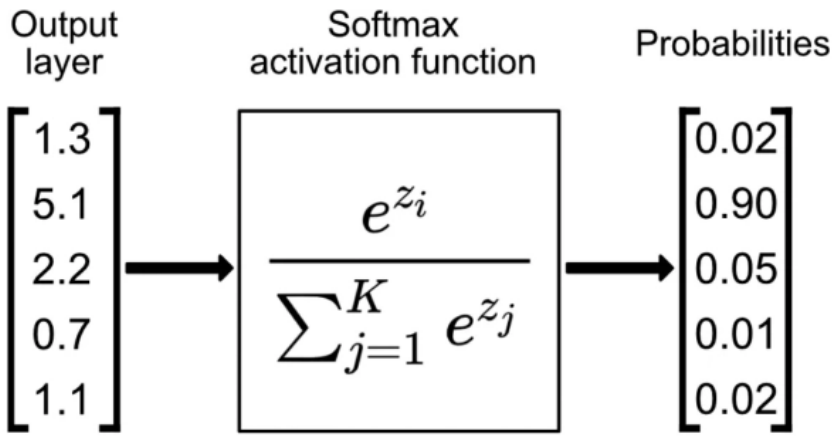
Softmax Function

The softmax function is also a type of sigmoid function but is handy when we are trying to **handle multi- class classification problems.**

Nature :- non-linear

Uses :- Usually used when trying to handle multiple classes. the softmax function was commonly found in the output layer of image classification problems. The softmax function would squeeze the outputs for each class between 0 and 1 and would also divide by the sum of the outputs.

Output:- The softmax function is ideally used in the output layer of the classifier where we are actually trying to attain the probabilities to define the class of each input.



- The basic rule of thumb is if you really don't know what activation function to use, then simply use *RELU* as it is a general activation function in hidden layers and is used in most cases these days.
- If your output is for **binary classification** then, ***sigmoid function*** is very natural choice for output layer.
- If your output is for **multi-class classification** then, **Softmax** is very useful to predict the probabilities of each classes.

RELU Function

- It Stands for *Rectified linear unit*. It is the most widely used activation function. Chiefly implemented in *hidden layers* of Neural network.
- **Equation :-** ***A(x) = max(0,x)***. It gives an output x if x is positive and 0 otherwise.
- **Value Range :-** [0, inf)
- **Nature :-** non-linear, which means we can easily backpropagate the errors and have multiple layers of neurons being activated by the ReLU function.
- **Uses :-** ReLu is less computationally expensive than tanh and sigmoid because it involves simpler mathematical operations. At a time only a few neurons are activated making the network sparse making it efficient and easy for computation.

In simple words, RELU learns *much faster* than sigmoid and Tanh function.

Numerical

Let's go through a simple numerical example using the **sigmoid activation function**. The sigmoid function is given by:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Example Problem:

Given an input $x = 2$, calculate the output of the sigmoid activation function.

Solution

1. **Substitute x into the sigmoid function:**

$$\sigma(2) = \frac{1}{1 + e^{-2}}$$

2. **Calculate the exponent e^{-2}:** Using the approximate value $e \approx 2.718$:

$$e^{-2} = \frac{1}{e^2} \approx \frac{1}{2.718^2} \approx \frac{1}{7.389} \approx 0.135$$

3. **Add 1 to the result:**

$$1 + 0.135 = 1.135$$

4. **Take the reciprocal:**

$$\frac{1}{1.135} \approx 0.880$$

Numerical

Let's try a simple numerical example using the **ReLU (Rectified Linear Unit)** activation function.

The ReLU function is defined as:

$$\text{ReLU}(x) = \max(0, x)$$

Example Problem:

Given two inputs $x_1 = 3$ and $x_2 = -1.5$, calculate the output of the ReLU activation function for both inputs.

Solution

For $x_1 = 3$:

1. Apply the ReLU function:

$$\text{ReLU}(3) = \max(0, 3)$$

2. Calculate the maximum value:

$$\max(0, 3) = 3$$

Output for $x_1 = 3$: 3

For $x_2 = -1.5$:

1. Apply the ReLU function:

$$\text{ReLU}(-1.5) = \max(0, -1.5)$$

2. Calculate the maximum value:

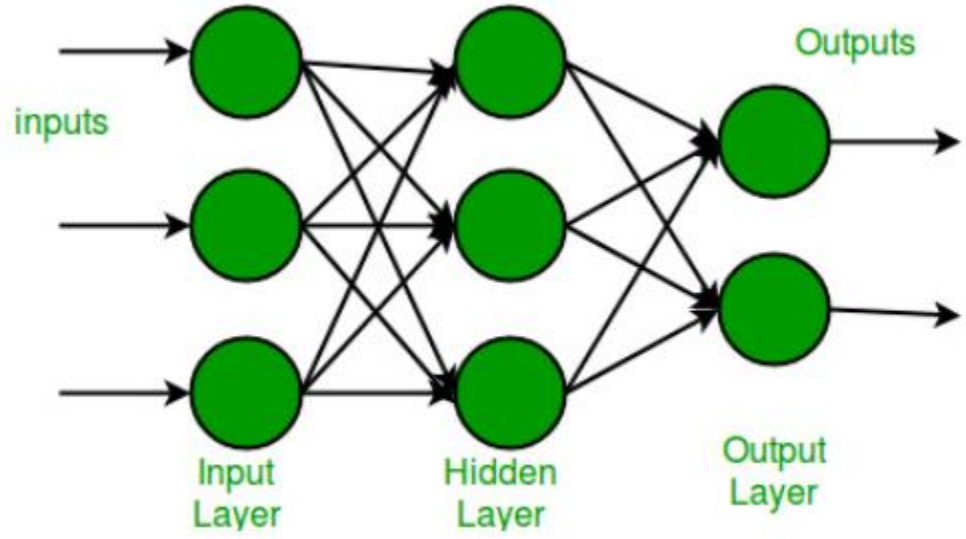
$$\max(0, -1.5) = 0$$

Output for $x_2 = -1.5$: 0

Multilayer Perceptron

- Multi-layer perception is also known as MLP. It is fully connected dense layers, which transform any input dimension to the desired dimension. A multi-layer perception is a neural network that has multiple layers. To create a neural network we combine neurons together so that the outputs of some neurons are inputs of other neurons.

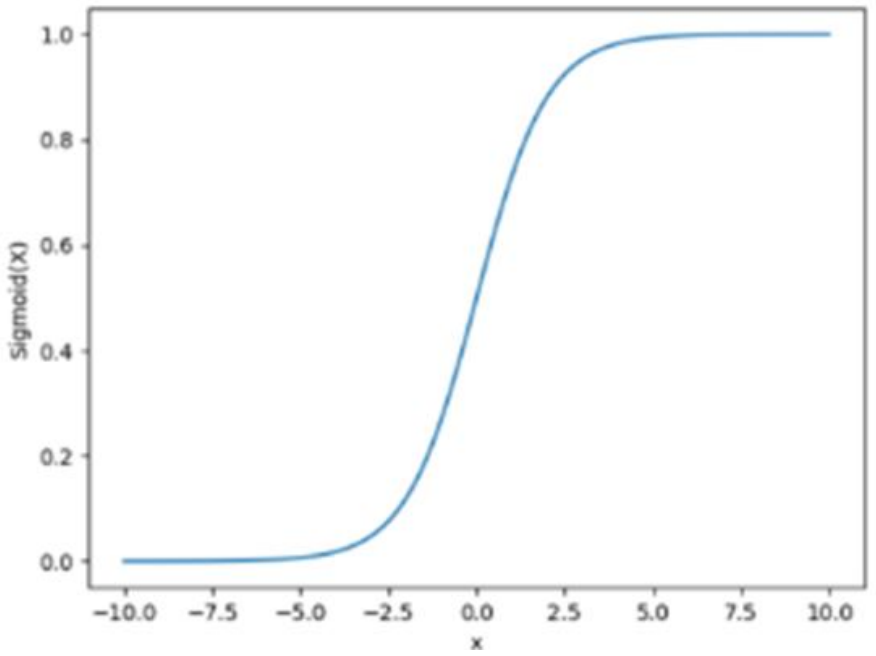
- A multi-layer perceptron has one input layer and for each input, there is one neuron(or node), it has one output layer with a single node for each output and it can have any number of hidden layers and each hidden layer can have any number of nodes. A schematic diagram of a Multi-Layer Perceptron (MLP) is depicted below.



- In the multi-layer perceptron diagram, we can see that there are three inputs and thus three input nodes and the hidden layer has three nodes. The output layer gives two outputs, therefore there are two output nodes. The nodes in the input layer take input and forward it for further process, in the diagram above the nodes in the input layer forwards their output to each of the three nodes in the hidden layer, and in the same way, the hidden layer processes the information and passes it to the output layer.

- Every node in the multi-layer perception uses a **sigmoid activation function**. The sigmoid activation function takes real values as input and converts them to numbers between 0 and 1 using the sigmoid formula.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Structure of MLP:

- **Input Layer**: This layer receives the input features.
- **Hidden Layer(s)**: One or more layers that process inputs with learned weights and biases, followed by activation functions. Each hidden layer is fully connected to the previous layer.
- **Output Layer**: Provides the output predictions. The number of neurons in this layer depends on the task (e.g., one neuron for binary classification, multiple neurons for multi-class classification).

Key Components:

- **Weights**: Parameters associated with connections between neurons, adjusted during training.
- **Biases**: Additional parameters added to the weighted sum of inputs before passing through the activation function.
- **Activation Function**: Introduces non-linearity into the model, enabling it to capture complex patterns. Common activation functions include:
  - ReLU (Rectified Linear Unit)
  - Sigmoid (for binary classification)
  - Softmax (for multi-class classification)

- **Forward Pass:** The data is fed forward through the network from the input layer, through the hidden layers, to the output layer. Each layer applies a weighted sum, adds biases, and then passes the result through an activation function.

- **Backpropagation:** MLP uses **backpropagation** for training. The difference between the predicted and actual values (error) is propagated backward through the network, adjusting weights and biases using gradient descent.

Use cases

- **Classification**: MLP can be used for tasks like image recognition, speech recognition, and spam detection.

- **Regression**: It can predict continuous values by adjusting the output layer and loss function.

Question: If a neural network has an input layer with 3 neurons, one hidden layer with 4 neurons, and an output layer with 2 neurons, calculate the following:

- a) Number of parameters (weights and biases) if using sigmoid activation in the hidden layer.
- b) Number of parameters if using ReLU activation in the hidden layer (assuming biases are included in both cases).

Answer:

1. **Weights from Input to Hidden Layer:**
   - 3 input neurons * 4 hidden neurons = 12 weights
2. **Biases for Hidden Layer:**
   - 4 biases (one for each hidden neuron)
3. **Weights from Hidden to Output Layer:**
   - 4 hidden neurons * 2 output neurons = 8 weights
4. **Biases for Output Layer:**
   - 2 biases (one for each output neuron)

Total Parameters (Weights + Biases) = 12 + 4 + 8 + 2 = **26**

Note: The activation function (Sigmoid or ReLU) does not affect the count of weights and biases, so the answer is **26 parameters** for both cases.

Numerical

Problem Setup:

We will use the following simple neural network:

- **Input layer**: 2 neurons (inputs x_1 and x_2)
- **Hidden layer**: 1 neuron, with ReLU activation
- **Output layer**: 1 neuron, with sigmoid activation

Given:

- Input vector: $\mathbf{x} = [1, 1]$
- Weight between input and hidden layer:

$$\mathbf{W_1} = [0.5, 0.5]$$

- Bias for hidden layer: $b_1 = 0.1$
- Weight between hidden and output layer:

$$\mathbf{W_2} = 1.0$$

- Bias for output layer: $b_2 = 0.0$

Solution

1. Compute Hidden Layer Output:

$$z_1 = (\mathbf{W_1} \cdot \mathbf{x}) + b_1$$

Substitute the values:

$$z_1 = (0.5 \cdot 1) + (0.5 \cdot 1) + 0.1$$

$$z_1 = 0.5 + 0.5 + 0.1 = 1.1$$

Apply the ReLU activation function:

$$\text{ReLU}(z_1) = \max(0, 1.1) = 1.1$$

2. Compute Output Layer:

$$z_2 = (W_2 \cdot \text{ReLU}(z_1)) + b_2$$

Substitute the values:

$$z_2 = (1.0 \cdot 1.1) + 0.0 = 1.1$$

Apply the sigmoid activation function:

$$\sigma(z_2) = \frac{1}{1 + e^{-1.1}}$$

Approximate $e^{-1.1} \approx 0.333$:

$$\sigma(1.1) = \frac{1}{1 + 0.333} \approx \frac{1}{1.333} \approx 0.75$$

Gradient Descent

- Gradient descent (GD) is an iterative first-order optimisation algorithm, used to find a local minimum/maximum of a given function.
- Commonly used in machine learning (ML) and deep learning (DL) to minimise a cost/loss function (e.g. in a linear regression).

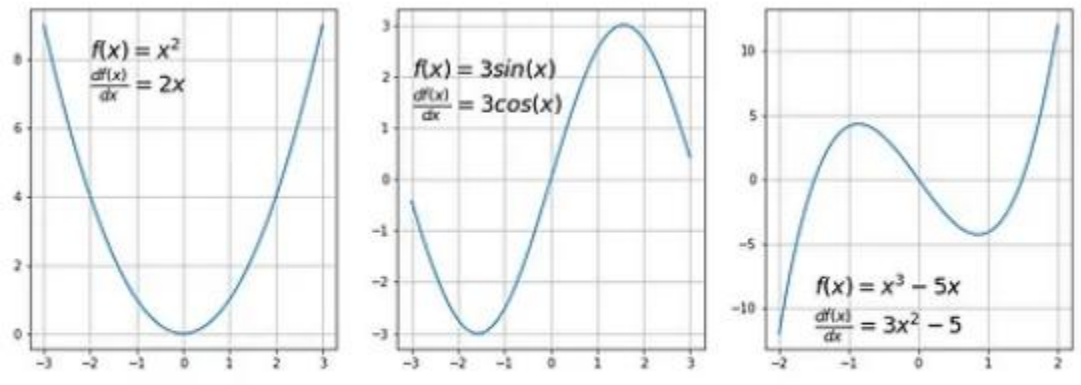
Function requirements

Gradient descent algorithm does not work for all functions. There are two specific requirements. A function has to be:

1. Differentiable
2. Convex

what does it mean it has to be differentiable?

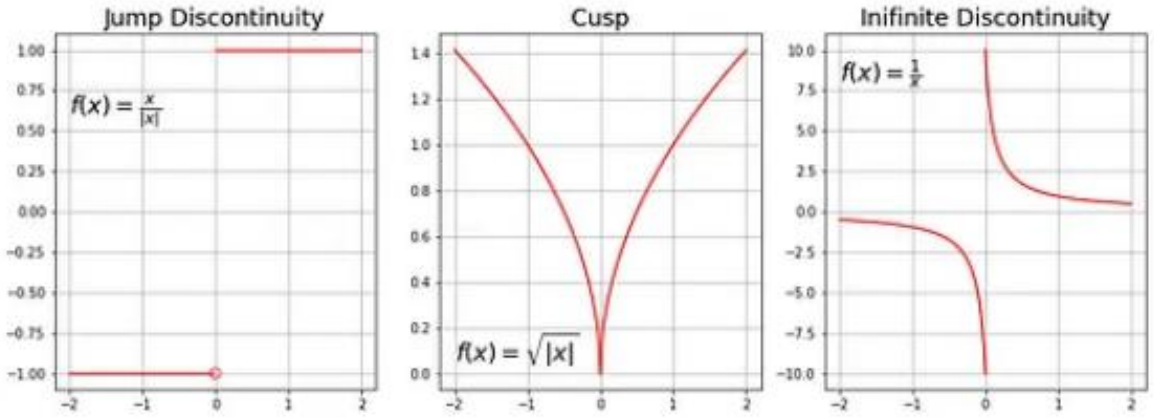
- If a function is differentiable it has a derivative for each point in its domain — not all functions meet these criteria.



Examples of differentiable functions; Image by author

Non differentiable Functions

Typical non-differentiable functions have a step a cusp or a discontinuity:

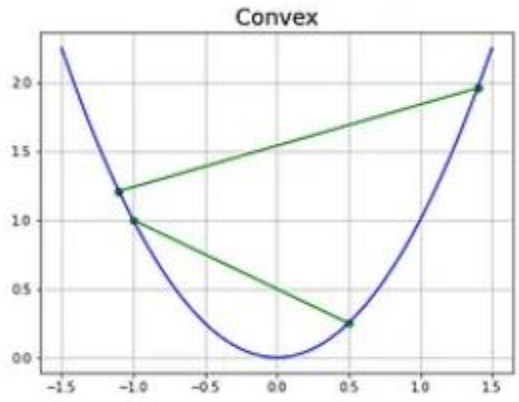


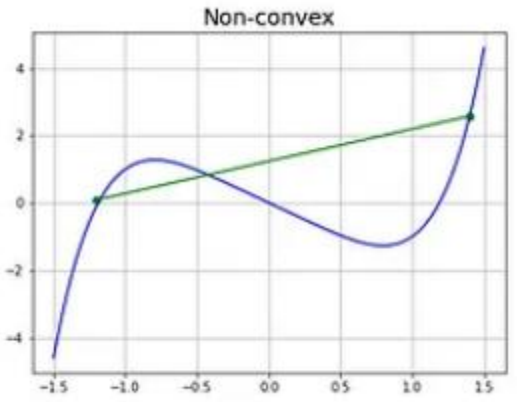
Examples of non-differentiable functions; Image by author

Next requirement of Gradient Descent — function has to be convex.

- For a univariate function, this means that the line segment connecting two function's points lays on or above its curve (it does not cross it). If it does it means that it has a local minimum which is not a global one.
- Another way to check mathematically if a univariate function is convex is to calculate the second derivative and check if its value is always bigger than 0.

$$\frac{d^2 f(x)}{dx^2} > 0$$





Numerical

Check whether these functions are convex or Not

1. $f(x) = x^2 - x + 3$

2. $f(x) = x^4 - 2x^3 + 2$

1.

$$f(x) = x^2 - x + 3$$

$$\frac{df(x)}{dx} = 2x - 1, \quad \frac{d^2 f(x)}{dx^2} = 2$$

Because the second derivative is always bigger than 0, our function is strictly convex.

2.

$$f(x) = x^4 - 2x^3 + 2$$

$$\frac{df(x)}{dx} = 4x^3 - 6x^2 = x^2(4x - 6)$$

$$\frac{d^2 f(x)}{dx^2} = 12x^2 + 12x = 12x(x - 1)$$

The value of this expression is zero for *x=0* and *x=1*. These locations are called an inflexion point — a place where the curvature changes sign — meaning it changes from convex to concave or vice-versa. By analysing this equation we conclude that :

- for x<0: function is convex
- for 0<x<1: function is concave (the 2nd derivative < 0)
- for x>1: function is convex again

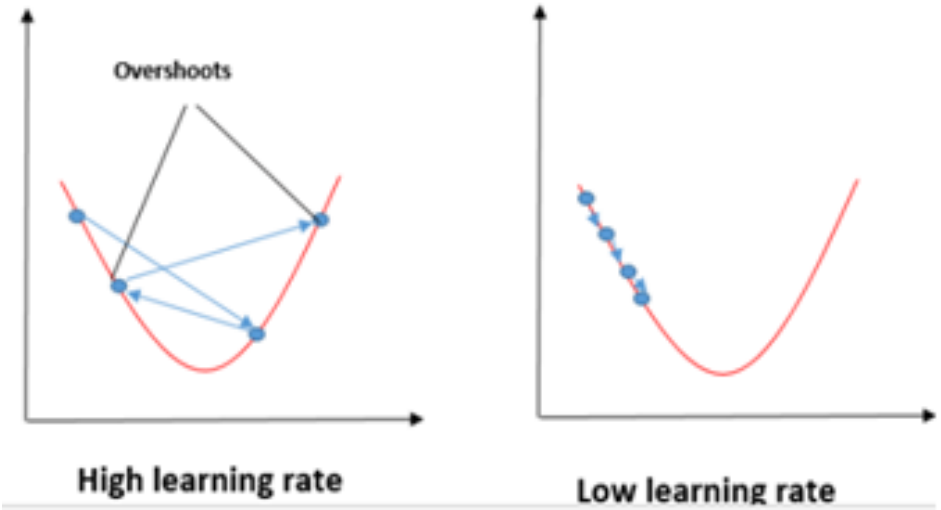
Gradient Descent Algorithm

- Gradient Descent Algorithm iteratively calculates the next point using gradient at the current position, scales it (by a learning rate) and subtracts obtained value from the current position (makes a step). It subtracts the value because we want to minimise the function (to maximise it would be adding).
- This process can be written as:

$$p_{n+1} = p_n - \eta \nabla f(p_n)$$

Learning Rate

- There's an important parameter η which scales the gradient and thus controls the step size.
- In machine learning, it is called learning rate and have a strong influence on performance.
  - The smaller learning rate the longer GD converges, or may reach maximum iteration before reaching the optimum point
  - If learning rate is too big the algorithm may not converge to the optimal point (jump around) or even to diverge completely.



Steps of Algorithm

1. Initialize Parameters:

Start with an initial guess for the model parameters (e.g., weights, biases).

2. Compute the Gradient:

- Calculate the gradient of the cost function with respect to each parameter. The gradient indicates the direction of the steepest increase in the cost function.

3. **Update the Parameters:**

   - Adjust the parameters in the opposite direction of the gradient to reduce the cost function:

     $$\theta = \theta - \alpha \nabla J(\theta)$$

     Where:

     - θ represents the parameters.
     - α is the learning rate (a small positive number).
     - $\nabla J(\theta)$ is the gradient of the cost function $J(\theta)$ with respect to θ.

4. **Repeat:**

   - Repeat the process until convergence, which occurs when the gradient becomes small or the cost function changes by a very small amount between iterations.

Types of Gradient Descent:

1. **Batch Gradient Descent:**
    - Uses the entire dataset to compute the gradient for each update.
    - Converges smoothly but can be slow for large datasets.
2. **Stochastic Gradient Descent (SGD):**
    - Uses only one sample at each iteration to compute the gradient.
    - Faster for large datasets but introduces more noise into the updates.
3. **Mini-batch Gradient Descent:**
    - Uses a small random subset (mini-batch) of the dataset to compute the gradient.
    - Balances the trade-offs between batch and stochastic gradient descent.

Numerical

Minimize the function $f(x) = (x-2)^2$.

Initial Setup:

- Start with $x = 0$.
- Learning rate $\alpha = 0.1$.

Solution

Function and its Derivative:

- $f(x) = (x-2)^2$
- Derivative: $f'(x) = 2(x-2)$

Initial Setup:

- Start with $x = 0$.
- Learning rate $\alpha = 0.1$.

Gradient Descent Formula:

- $x_{\text{new}} = x_{\text{old}} - \alpha f'(x_{\text{old}})$

1. Iteration 1:
   - $x_{\text{old}} = 0$
   - $f'(x) = 2(0-2) = -4$
   - $x_{\text{new}} = 0 - 0.1 \times (-4) = 0.4$
2. Iteration 2:
   - $x_{\text{old}} = 0.4$
   - $f'(x) = 2(0.4-2) = -3.2$
   - $x_{\text{new}} = 0.4 - 0.1 \times (-3.2) = 0.72$
3. Iteration 3:
   - $x_{\text{old}} = 0.72$
   - $f'(x) = 2(0.72-2) = -2.56$
   - $x_{\text{new}} = 0.72 - 0.1 \times (-2.56) = 0.976$

Numerical

Consider the following simple linear regression model:

$$y = w \cdot x + b$$

You are given a dataset with a single data point $(x, y) = (2, 5)$. The loss function is the Mean Squared Error (MSE), defined as:

$$L(w, b) = \frac{1}{2} \times (y_{\text{predicted}} - y)^2$$

Where $y_{\text{predicted}} = w \cdot x + b$.

Given the initial values $w = 0$, $b = 0$, and a learning rate $\eta = 0.1$, perform one iteration of gradient descent to update w and b. Show the gradients, the updated values of w and b, and the loss after the update.

Solution

Given:

- Data point: $(x, y) = (2, 5)$
- Initial values: $w = 0, b = 0$
- Learning rate: $\eta = 0.1$
- Loss function (MSE):

$$L(w, b) = \frac{1}{2} \times (y_{\text{predicted}} - y)^2$$

where $y_{\text{predicted}} = w \cdot x + b$.

Step 1: Compute $y_{\text{predicted}}$

Using the initial values of w and b:

$$y_{\text{predicted}} = (0 \cdot 2) + 0 = 0$$

Step 2: Compute the Loss

$$L(w, b) = \frac{1}{2} \times (0 - 5)^2 = \frac{1}{2} \times 25 = 12.5$$

Step 3: Compute the Gradients

The gradients of the loss function with respect to w and b are:

1. **Gradient with respect to w:**

$$\frac{\partial L}{\partial w} = (y_{\text{predicted}} - y) \cdot x = (0 - 5) \cdot 2 = -10$$

2. **Gradient with respect to b:**

$$\frac{\partial L}{\partial b} = (y_{\text{predicted}} - y) = 0 - 5 = -5$$

Step 4: Update w and b

Using the gradient descent update rules:

$$w \leftarrow w - \eta \times \frac{\partial L}{\partial w}$$

$$b \leftarrow b - \eta \times \frac{\partial L}{\partial b}$$

Substituting the values:

$$w \leftarrow 0 - 0.1 \times (-10) = 0 + 1 = 1$$

$$b \leftarrow 0 - 0.1 \times (-5) = 0 + 0.5 = 0.5$$

Step 5: Compute the New Loss

Now, compute the loss with updated w and b:

$$y_{\text{predicted}} = (1 \cdot 2) + 0.5 = 2.5$$

$$L(w, b) = \frac{1}{2} \times (2.5 - 5)^2 = \frac{1}{2} \times (-2.5)^2 = \frac{1}{2} \times 6.25 = 3.125$$

Final Updated Values:

- $w = 1$
- $b = 0.5$
- New Loss: 3.125

This completes one iteration of gradient descent.

Chain Rule

The **chain rule** is a fundamental concept in backpropagation, which is used to compute the gradients of the loss function with respect to each weight in a neural network. Backpropagation allows neural networks to efficiently learn by updating weights using gradient descent.

Understanding the Chain Rule

In calculus, the chain rule states:

$$\frac{dy}{dx} = \frac{dy}{du} \times \frac{du}{dx}$$

This applies when y is a function of u, which in turn is a function of x.

Application in Backpropagation

For a single layer L:

Let:

- $a^{[L]}$ be the output of layer L.
- $z^{[L]} = w^{[L]}a^{[L-1]} + b^{[L]}$ (weighted sum before activation).
- $a^{[L]} = f(z^{[L]})$ where f is the activation function.

If the loss function is J, we seek $\frac{\partial J}{\partial w^{[L]}}$.

Step-by-Step Using Chain Rule:

1. Compute $\frac{\partial J}{\partial a^{[L]}}$.
2. Compute $\frac{\partial a^{[L]}}{\partial z^{[L]}}$ using the derivative of the activation function f.
3. Compute $\frac{\partial z^{[L]}}{\partial w^{[L]}}$.

$$\frac{\partial J}{\partial w^{[L]}} = \frac{\partial J}{\partial z^{[L]}} \times \frac{\partial z^{[L]}}{\partial w^{[L]}}$$

Where:

- $\frac{\partial z^{[L]}}{\partial w^{[L]}} = a^{[L-1]}$

Why the Chain Rule is Essential

- Neural networks are composed of nested functions.
- The chain rule helps compute derivatives efficiently for these nested functions.
- Without the chain rule, calculating gradients for deep networks would be computationally infeasible.

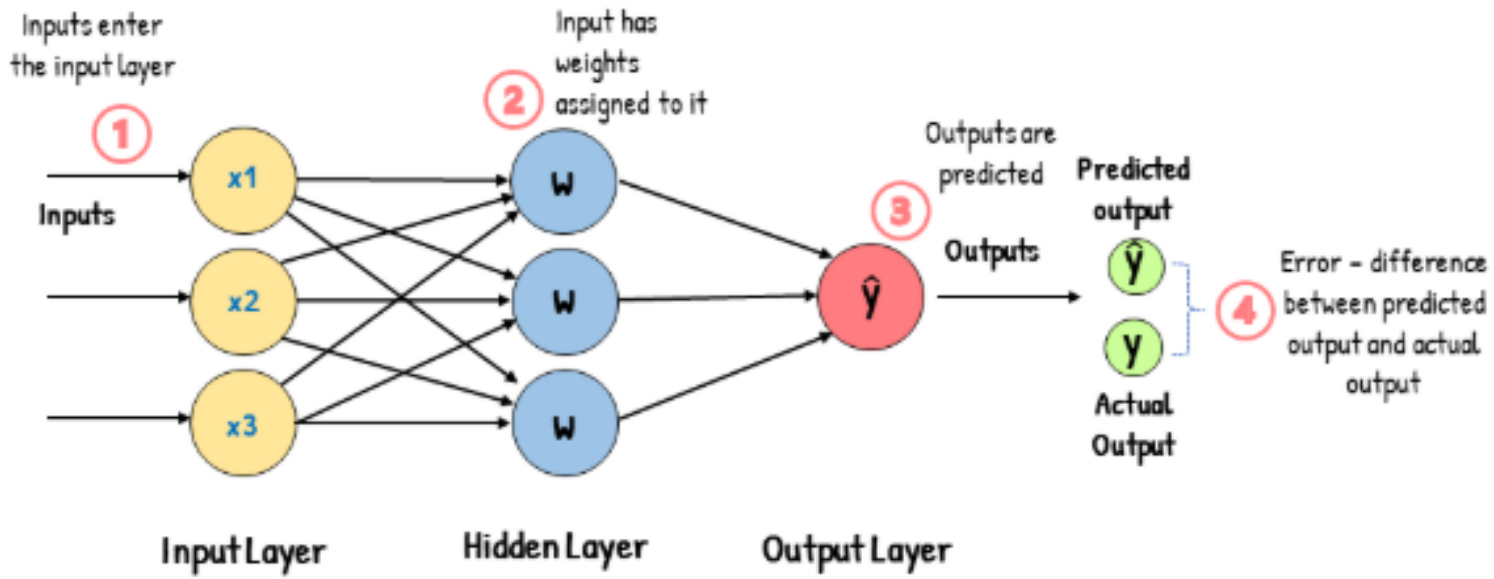
Backpropagation Learning Algorithm

Back propagation, often simply referred to as back propagation, is a widely used algorithm in the training of neural networks for supervised learning. **Backpropagation efficiently computes the gradient of the loss function with respect to the weights of the network.** This gradient is then used by an optimization algorithm, such as stochastic gradient descent, to adjust the weights to minimize the loss.

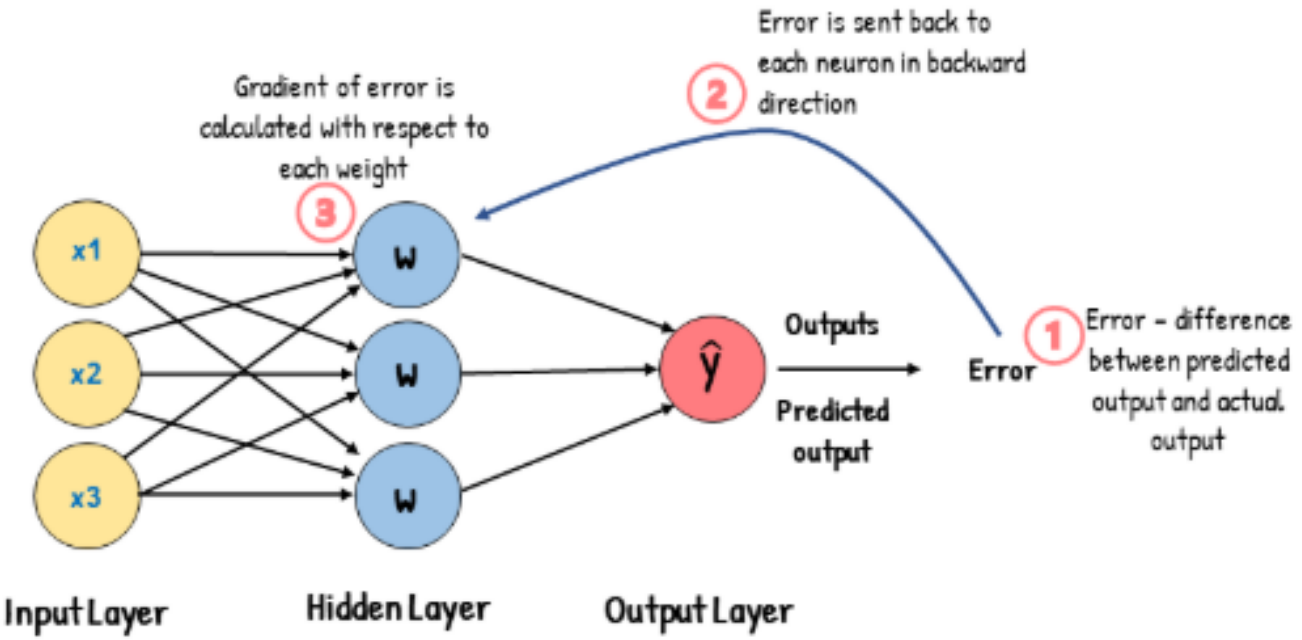
The process involves two main phases:

- a forward pass, where the input data is passed through the network to compute the output,
- a backward pass, where the gradients are computed by propagating the error backward through the network.

Feed-Forward Neural Network



Backpropagation



Main Steps: Back Propagation Algorithm

Backpropagation is used to train neural networks by adjusting weights to minimize the error between predicted and actual output. It involves the following steps:

1. Forward Propagation:

- Compute the output of the network using current weights.

2. Compute Loss:

- Calculate the difference between the predicted and actual output (loss function).

3. Backward Propagation:

- Compute the gradient of the loss with respect to each weight using the chain rule.
- Update the weights in the direction that reduces the loss.

4. Update Weights:

- Adjust weights using the calculated gradients and a learning rate.

Back Propagation Algorithm

1. Forward Pass:

Compute the output of the network:

1. **Inputs** $(x_1, x_2, \ldots, x_n)$ are passed to the input layer.
2. Each neuron calculates its **weighted sum**:

$$z = \sum(w \cdot x) + b$$

   where:
   - w: weights
   - x: inputs
   - b: bias
3. Apply the **activation function** (e.g., sigmoid, ReLU):

$$a = \text{activation}(z)$$

4. Repeat for all layers until the output.

2. Compute the Loss:

Evaluate the loss function to measure the difference between the predicted output $\hat{y}$ and the actual target y:

$$\text{Loss} = \frac{1}{2}(y - \hat{y})^2 \quad \text{(for mean squared error)}$$

3. Backward Pass:

Step 1: Output Layer Error

For the output neuron, calculate the gradient of the loss with respect to the output activation:

$$\delta_{\text{output}} = \frac{\partial \text{Loss}}{\partial a_{\text{output}}} \cdot \frac{\partial a_{\text{output}}}{\partial z_{\text{output}}}$$

- First term: $\frac{\partial \text{Loss}}{\partial a_{\text{output}}} = (\hat{y} - y)$
- Second term (for sigmoid): $\frac{\partial a_{\text{output}}}{\partial z_{\text{output}}} = a_{\text{output}}(1 - a_{\text{output}})$

Step 2: Hidden Layer Error

For hidden neurons, compute the gradient using the chain rule:

$$\delta_{\text{hidden}} = \frac{\partial a_{\text{hidden}}}{\partial z_{\text{hidden}}} \cdot \sum (\delta_{\text{next}} \cdot w_{\text{next}})$$

- First term: Derivative of activation function (e.g., sigmoid: $a(1-a)$).
- Second term: Weighted sum of the errors from the next layer.

Step 3: Gradients for Weights and Biases

Update the gradients for each weight and bias:

- Weight gradients:

$$\frac{\partial \text{Loss}}{\partial w} = \delta \cdot a_{\text{previous}}$$

- Bias gradients:

$$\frac{\partial \text{Loss}}{\partial b} = \delta$$

4. Update Weights and Biases:

Using gradient descent:

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \frac{\partial \text{Loss}}{\partial w}$$

$$b_{\text{new}} = b_{\text{old}} - \eta \cdot \frac{\partial \text{Loss}}{\partial b}$$

where η is the learning rate.

5. Iterate:

Repeat the forward and backward passes for a set number of epochs or until convergence.

Derivative of the Sigmoid Function

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

Step 1: Differentiating both sides with respect to x.

$$F'(x) = \frac{d}{dx}\left(\frac{1}{1+e^{-x}}\right)$$

Step 2: Apply the reciprocating/chain rule.

$$\begin{aligned} F'(x) &= \frac{d}{dx}\left(\frac{1}{1+e^{-x}}\right) \\ &= -\frac{1}{(1+e^{-x})^2}\frac{d}{dx}(1+e^{-x}) \\ &= -\frac{1}{(1+e^{-x})^2}\cdot e^{-x}\frac{d}{dx}(-x) \\ &= \frac{e^{-x}}{(1+e^{-x})^2} \end{aligned}$$

Step 3: Modify the equation for a more generalized form.

$$
\begin{aligned}
\sigma^{'}(x) &= \frac{e^{-x}}{\left(1+e^{-x}\right)^2} \\
&= \frac{1+e^{-x}-1}{\left(1+e^{-x}\right)^2} \\
&= \frac{1}{1+e^{-x}} - \frac{1}{\left(1+e^{-x}\right)^2} \\
&= \frac{1}{\left(1+e^{-x}\right)} \cdot \left(1 - \frac{1}{1+e^{-x}}\right) \\
&= \sigma(x)\left(1-\sigma(x)\right)
\end{aligned}
$$

Numerical

We have a simple neural network with:

- **Input (x):** $x = 1.0$
- **Weight (w):** $w = 0.5$
- **Bias (b):** $b = 0.0$
- **Target (y):** $y = 0.8$
- **Activation Function:** Sigmoid.

We'll calculate the **forward pass**, **loss**, and then perform the **backward pass** using the **chain rule**.

Solution

1. Forward Pass

Step 1: Calculate weighted sum (z):

$$z = w \cdot x + b$$

$$z = 0.5 \cdot 1.0 + 0.0 = 0.5$$

Step 2: Apply sigmoid activation:

The sigmoid function is:

$$a = \frac{1}{1 + e^{-z}}$$

Substituting $z = 0.5$:

$$a = \frac{1}{1 + e^{-0.5}} \approx 0.6225$$

Step 3: Compute the loss:

We'll use Mean Squared Error (MSE):

$$\text{Loss} = \frac{1}{2}(y - a)^2$$

Substituting $y = 0.8$ and $a = 0.6225$:

$$\text{Loss} = \frac{1}{2}(0.8 - 0.6225)^2 = \frac{1}{2}(0.1775)^2 \approx 0.0158$$

2. Backward Pass

Now, let's compute the gradients for backpropagation.

Step 1: Gradient of the Loss w.r.t. Output Activation (a)

The derivative of the loss with respect to a is:

$$\frac{\partial \text{Loss}}{\partial a} = a - y$$

Substituting $a = 0.6225$ and $y = 0.8$:

$$\frac{\partial \text{Loss}}{\partial a} = 0.6225 - 0.8 = -0.1775$$

Step 2: Gradient of the Activation w.r.t. Weighted Sum (z)

For the sigmoid function, the derivative is:

$$\frac{\partial a}{\partial z} = a(1 - a)$$

Substituting $a = 0.6225$·

$$\frac{\partial a}{\partial z} = 0.6225 \cdot (1 - 0.6225) \approx 0.2350$$

Step 3: Chain Rule for Gradient of the Loss w.r.t. z

Using the chain rule:

$$\frac{\partial \text{Loss}}{\partial z} = \frac{\partial \text{Loss}}{\partial a} \cdot \frac{\partial a}{\partial z}$$

Substituting:

$$\frac{\partial \text{Loss}}{\partial z} = (-0.1775) \cdot (0.2350) \approx -0.0417$$

Step 4: Gradient of the Loss w.r.t. Weight (w)

Using the chain rule, the gradient of the loss with respect to w is:

$$\frac{\partial \text{Loss}}{\partial w} = \frac{\partial \text{Loss}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Since $\frac{\partial z}{\partial w} = x$:

$$\frac{\partial \text{Loss}}{\partial w} = (-0.0417) \cdot 1.0 = -0.0417$$

Step 5: Gradient of the Loss w.r.t. Bias (b)

Using the chain rule, the gradient of the loss with respect to b is:

$$\frac{\partial \text{Loss}}{\partial b} = \frac{\partial \text{Loss}}{\partial z} \cdot \frac{\partial z}{\partial b}$$

Since $\frac{\partial z}{\partial b} = 1$:

$$\frac{\partial \text{Loss}}{\partial b} = (-0.0417) \cdot 1 = -0.0417$$

3. Update Parameters

Using gradient descent with a learning rate $\eta = 0.1$:

Update weight (w):

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \frac{\partial \text{Loss}}{\partial w}$$

$$w_{\text{new}} = 0.5 - 0.1 \cdot (-0.0417) = 0.5 + 0.00417 = 0.5042$$

Update bias (b):

$$b_{\text{new}} = b_{\text{old}} - \eta \cdot \frac{\partial \text{Loss}}{\partial b}$$

$$b_{\text{new}} = 0.0 - 0.1 \cdot (-0.0417) = 0.0 + 0.00417 = 0.0042$$

Final Updated Parameters:

- **Weight:** $w = 0.5042$
- **Bias:** $b = 0.0042$

Challenges with Backpropagation

- **Vanishing Gradients:** In deep networks, the gradients can become very small, effectively preventing the weights in the earlier layers from changing significantly. This can slow down or even halt training.
- **Exploding Gradients:** Conversely, gradients can also grow exponentially, causing large weight updates that can destabilize the learning process.
- **Non-Convex Loss Functions:** Neural networks typically have non-convex loss functions, which means there can be multiple local minima. Gradient descent methods can get stuck in these local minima instead of finding the global minimum.

How to address these challenges

To address these challenges, various improvements and variants of the backpropagation algorithm have been developed:

- **Activation Functions:** Functions like ReLU and its variants help mitigate the vanishing gradient problem.
- **Weight Initialization:** Techniques like Xavier and He initialization set the initial weights to values that help prevent vanishing or exploding gradients.
- **Gradient Clipping:** Limits the size of gradient. This technique prevents gradients from becoming too large, addressing the exploding gradient problem.
- **Optimization Algorithms:** Advanced optimizers like Adam, RMSprop, and AdaGrad adapt the learning rate during training to improve convergence.

Empirical Risk Minimization

- Empirical Risk Minimization (ERM) is a fundamental concept in machine learning used for training models by minimizing the average loss over a given dataset. It provides a formal framework to select the model parameters that best fit the training data.

- We have a training dataset $\{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$, where:
  - x_i are input features.
  - y_i are true outputs/labels.
- We want to train a model $f(x; \theta)$ that predicts y for a given x, where θ represents the model's parameters.

- **Loss Function**:
  - Define a loss function $\mathcal{L}(y, f(x;\theta))$ that measures how far the predicted value $f(x;\theta)$ is from the true value y.
- **Empirical Risk**:
  - The empirical risk is the average loss across all n training examples:

$$R_{emp}(\theta) = \frac{1}{n}\sum_{i=1}^{n}\mathcal{L}(y_i, f(x_i;\theta)).$$

- **Goal**:
  - Find the parameter θ that minimizes the empirical risk:

$$\theta^* = \arg\min_{\theta} R_{emp}(\theta).$$

Challenges in ERM:

1. **Overfitting :**Minimizing the empirical risk alone may lead to overfitting, where the model performs well the training data but poorly on unseen data. Regularization techniques, such as or regularization, are often used to mitigate this.
2. **Generalization:** ERM assumes the training data is representative of the entire data distribution. Generalization error measures the difference between the empirical risk and the true risk.
3. **Optimization:** Minimizing the empirical risk can be computationally intensive for complex models or large datasets.

Extensions of ERM:

1. **Regularized ERM**:
   - Adds a penalty term to the risk to control model complexity:

$$R_{reg}(\theta) = R_{emp}(\theta) + \lambda\Omega(\theta)$$

     where $\Omega(\theta)$ is the regularization term, and λ controls the strength of regularization.
2. **Stochastic ERM**:
   - Instead of using the entire dataset, stochastic gradient descent (SGD) is used to optimize the risk incrementally using mini-batches.

Numerical

Consider a binary classification problem where you have the following dataset:

Sample	x	True Class y	Predicted Class $\hat{y}$
1	0.5	1	0
2	1.2	0	0
3	-0.8	1	1
4	2.0	0	1
5	-1.5	1	1

The **0-1 loss function** is defined as:

$$L(y, \hat{y}) = \begin{cases} 0 & \text{if } y = \hat{y} \\ 1 & \text{if } y \neq \hat{y} \end{cases}$$

Find the empirical risk for this dataset.

Solution

Step 1: Compute the loss for each sample

- Sample 1: True $y = 1$, Predicted $\hat{y} = 0$ → Loss = 1
- Sample 2: True $y = 0$, Predicted $\hat{y} = 0$ → Loss = 0
- Sample 3: True $y = 1$, Predicted $\hat{y} = 1$ → Loss = 0
- Sample 4: True $y = 0$, Predicted $\hat{y} = 1$ → Loss = 1
- Sample 5: True $y = 1$, Predicted $\hat{y} = 1$ → Loss = 0

Step 2: Compute the empirical risk

The empirical risk is the average of all losses:

$$\text{Empirical Risk} = \frac{1}{n} \sum_{i=1}^{n} L(y_i, \hat{y}_i)$$

Substitute the values:

$$\text{Empirical Risk} = \frac{1 + 0 + 0 + 1 + 0}{5} = \frac{2}{5} = 0.4$$

Numerical

Suppose you have the following dataset for a simple regression problem:

x	y
1	3
2	5
3	7
4	9

The model is $f(x; w) = wx$, where w is the weight parameter. Use the Mean Squared Error (MSE) loss function defined as:

$$L(w) = \frac{1}{n} \sum_{i=1}^{n} (y_i - f(x_i; w))^2$$

Find the value of w that minimizes the empirical risk.

Solution

Step 1: Write the loss function

$$L(w) = \frac{1}{4}\sum_{i=1}^{4}(y_i - wx_i)^2$$

Step 2: Compute the partial derivative

$$\frac{dL(w)}{dw} = -\frac{2}{4}\sum_{i=1}^{4} x_i(y_i - wx_i)$$

Step 3: Set the derivative to 0 and solve for w

$$0 = \sum_{i=1}^{4} x_i(y_i - wx_i)$$

$$0 = \sum_{i=1}^{4}(x_i y_i) - w\sum_{i=1}^{4} x_i^2$$

Given:

$$\sum x_i y_i = 70 \quad \text{and} \quad \sum x_i^2 = 30$$

Thus:

$$70 = w \times 30$$

$$w = \frac{70}{30} = \frac{7}{3} \approx 2.33$$

Logistic Regression

- Logistic regression is a supervised learning algorithm which is mostly used for **binary classification** problems. Although "regression" contradicts with "classification", the focus here is on the word "logistic" referring to **logistic function** which does the classification task in this algorithm.
- Logistic regression is a simple yet very effective classification algorithm so it is commonly used for many binary classification tasks. Spam email, website or ad click predictions are some examples of the areas where logistic regression offers a powerful solution.

Logistic (Sigmoid) Function

- The basis of logistic regression is the logistic function, also called the sigmoid function, which takes in any real valued number and maps it to a value between 0 and 1.

$$Sigmoid\ Function:\ y = \frac{1}{1 + e^{-x}}$$

Logistic regression model takes a linear equation as input and use logistic function to perform a binary classification task.

How Does Logistic Regression work?

- Logistic regression model transforms the linear regression function continuous value output into categorical value output using a sigmoid function which maps any real-valued set of independent variables input into a value between 0 and 1. This function is known as the logistic function.

Suppose we have input features represented as a matrix:

$$X = \begin{bmatrix} x_{11} & \cdots & x_{1m} \\ x_{21} & \cdots & x_{2m} \\ \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{nm} \end{bmatrix}$$

and the dependent variable is Y having only binary value i.e 0 or 1.

$$Y = \begin{cases} 0 & \text{if } Class\ 1 \\ 1 & \text{if } Class\ 2 \end{cases}$$

then, apply the multi-linear function to the input variables X.

$$z = (\sum_{i=1}^{n} w_i x_i) + b$$

Here x_i is the ith observation of X, $w_i = [w_1, w_2, w_3, \cdots, w_m]$ is the weights or Coefficient and $b$is the bias term also known as intercept. Simply this can be represented as the dot product of weight and bias.

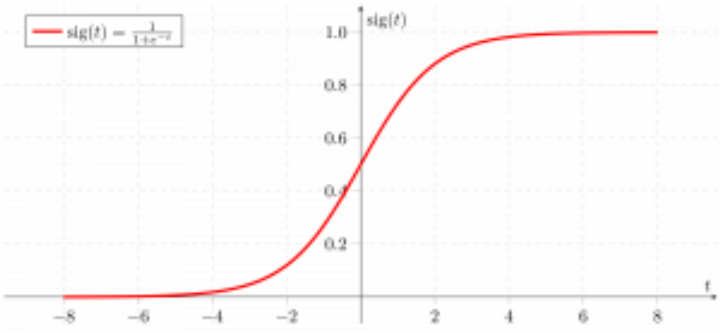
$$z = w \cdot X + b$$

At this stage, $z$is a continuous value from the linear regression. Logistic regression then applies the sigmoid function to $z$to convert it into a probability between 0 and 1 which can be used to predict the class.

Now we use the sigmoid function where the input will be z and we find the probability between 0 and 1. i.e. predicted y.

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

$$\sigma(z) = \frac{1}{1+e^{-z}}$$



Sigmoid function

As shown above the sigmoid function converts the continuous variable data into the probability i.e between 0 and 1.

- $\sigma(z)$ tends towards 1 as $z \to \infty$
- $\sigma(z)$ tends towards 0 as $z \to -\infty$
- $\sigma(z)$ is always bounded between 0 and 1

where the probability of being a class can be measured as:

$$P(y = 1) = \sigma(z)$$
$$P(y = 0) = 1 - \sigma(z)$$

Cost Function for Logistic Regression?

Why we can not use least square (mean squared error) cost function for logistic regression?

In **logistic regression**, the hypothesis (prediction) is defined as:

$$\hat{y} = \sigma(w^T x) = \frac{1}{1 + e^{-w^T x}}$$

where

- $x = [x_1, x_2, \ldots, x_n]$ is the input vector
- w is the weight vector
- $\sigma(\cdot)$ is the **sigmoid activation function**
- $\hat{y} \in (0, 1)$ represents the **predicted probability**
- $y \in \{0, 1\}$ is the **ground truth label**

Our objective is to choose a cost function $J(w)$ that penalizes wrong predictions effectively.

Using Least Squares Cost for Classification

A natural first thought is to use the **least squares (mean squared error)** cost:

$$J_{LS} = \frac{1}{2}(y - \hat{y})^2$$

This works well for **linear regression**, so the question is:

Can we use it for logistic regression?

Problem 1: Low Penalty for Confident Misclassification

Consider a binary classification example:

True label y	Predicted $\hat{y}$	MSE Loss
0	0.1	$\frac{1}{2}(0-0.1)^2 = 0.005$
0	0.9	$\frac{1}{2}(0-0.9)^2 = 0.405$
1	0.9	$\frac{1}{2}(1-0.9)^2 = 0.005$

issue:

Even when the model is **confidently wrong** (e.g., $y = 0$, $\hat{y} = 0.9$), the penalty is **not severe enough**.

For classification, **misclassification should be penalized heavily**, especially when confidence is high. MSE fails to do this.

Problem 2: Non-Convex Cost Function

When MSE is combined with the sigmoid function:

$$J(w) = \frac{1}{2}\left(y - \sigma(w^T x)\right)^2$$

the resulting cost function becomes **non-convex**.

Consequences:

- Multiple **local minima**
- Gradient descent may **not converge**
- Optimization becomes **slow and unstable**

This is undesirable, especially for large datasets.

Cost function for Logistic Regression: Log loss (cross-entropy loss):

To overcome these issues, we use the **log loss (cross-entropy loss)**:

$$J(w) = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right]$$

Why cross-entropy is better:

- Heavily penalizes confident misclassification
- Cost function is **convex**
- Faster and more reliable convergence

Numerical

Suppose we are trying to predict whether a student **passes (y=1)** or **fails (y=0)** an exam based on **hours studied (x)**.

We assume the logistic regression model:

$$p(y = 1|x) = \frac{1}{1 + e^{-(w_0 + w_1 x)}}$$

where:

- w_0 = intercept (bias)
- w_1 = weight (coefficient for hours studied)

Given:

- $w_0 = -4$
- $w_1 = 1$

We want to compute the probability of passing for:

1. Student who studied **3 hours** ($x = 3$)
2. Student who studied **6 hours** ($x = 6$)

Solution

$$z = w_0 + w_1 x$$

1. For $x = 3$:

$$z = -4 + 1(3) = -1$$

2. For $x = 6$:

$$z = -4 + 1(6) = 2$$

Apply sigmoid function

$$p = \frac{1}{1 + e^{-z}}$$

1. For $z = -1$:

$$p = \frac{1}{1 + e^{1}} = \frac{1}{1 + 2.718} \approx 0.268$$

So probability of passing = **26.8%**

2. For $z = 2$:

$$p = \frac{1}{1 + e^{-2}} = \frac{1}{1 + 0.135} \approx 0.881$$

So probability of passing = **88.1%**

- If $p \geq 0.5$ → predict **Pass (1)**
- If $p < 0.5$ → predict **Fail (0)**
- Student with **3 hours** → $p = 0.268 < 0.5$ → Predict **Fail**
- Student with **6 hours** → $p = 0.881 > 0.5$ → Predict **Pass**

Regularization

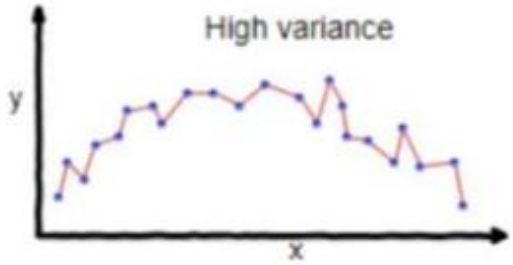
- Regularization techniques help prevent overfitting in deep learning models by introducing additional constraints or penalties during training.
- Overfitting occurs when a model performs well on training data but poorly on unseen data. Below are key regularization techniques commonly used:

Underfitting in Machine Learning

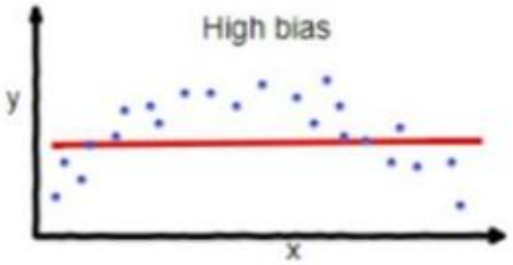
- Underfitting is the opposite of overfitting. It happens when a model is too simple to capture what's going on in the data.
- **Training Error:** High and **Test Error:** High
- High Bias

Overfitting in Machine Learning

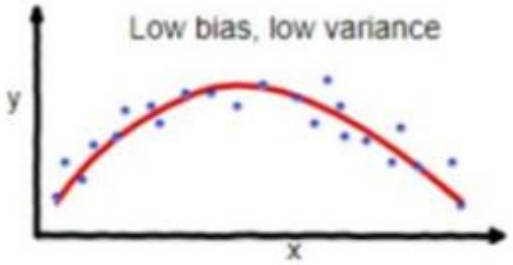
- Overfitting happens when a model learns too much from the training data, including details that don't matter (like noise or outliers).
- **Training Error:** Low and **Test Error:** High
- High Variance



overfitting

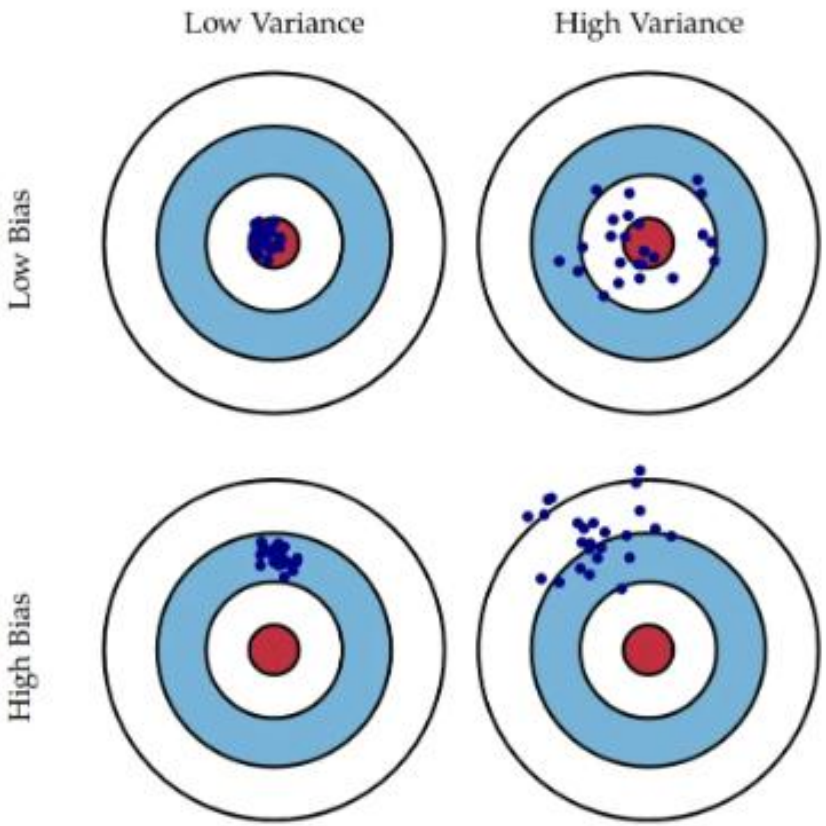


underfitting



Good balance

Bias and Variance



Bias in machine learning refers to the error introduced by approximating a real-world problem, which may be complex, using a simplified model. It arises when the assumptions made by the model are too rigid, leading to oversimplifications. Models with high bias tend to underfit the data, meaning they fail to capture the underlying patterns in the dataset.

Low Bias vs. High Bias

- **Low Bias**: A low-bias model generally fits the training data well because it can capture complex relationships in the data. However, this doesn't guarantee good performance on unseen data. A low-bias model might still overfit the training data if variance is too high.
- **High Bias**: A model with high bias tends to be overly simplistic, assuming a linear relationship when the data might be more complex. For example, if you're using a linear regression model for non-linear data, it could result in high bias, which leads to underfitting.

- **Variance** in machine learning refers to the model's sensitivity to small fluctuations in the training data. High variance occurs when a model captures not only the underlying patterns but also the noise in the training data, leading to overfitting. This means that while the model performs well on the training set, its performance on unseen data can degrade significantly.

Low Variance vs. High Variance

- **Low Variance**: A low-variance model tends to generalize well but may underfit the data if its bias is too high. Such a model may overlook complex relationships within the data.
- **High Variance**: High-variance models are prone to overfitting, as they adapt too closely to the specific data points in the training set, including noise. This leads to a model that performs well on the training data but struggles with generalization, meaning it will perform poorly on new data.

Regularization Techniques

- 1. L1 and L2 Regularization (Weight Decay)

These methods add penalties to the loss function based on the magnitude of the model weights:

- **L1 Regularization:** Adds the absolute value of the weights to the loss function. Encourages sparse models where some weights are exactly zero.

$$Loss = Loss_{original} + \lambda \sum |w_i|$$

- **L2 Regularization:** Adds the squared value of the weights to the loss function. Penalizes large weights, leading to smoother models.

$$Loss = Loss_{original} + \lambda \sum w_i^2$$

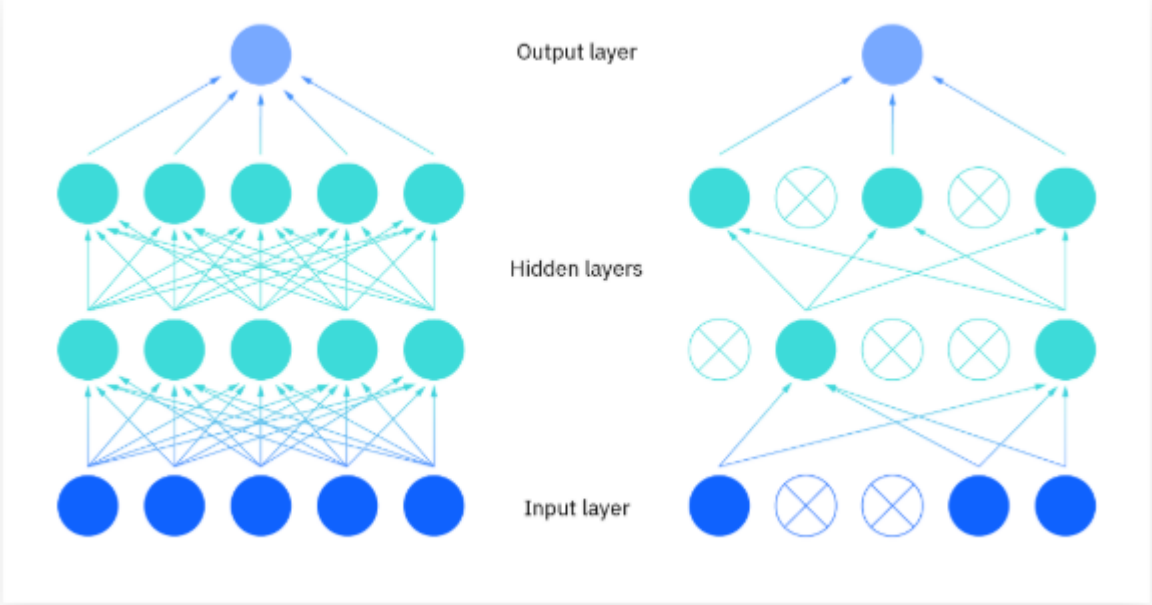
- Elastic Net: Combination of L1 and L2

2. Dropout

Dropout regularizes neural networks by randomly dropping out nodes, along with their input and output connections, from the network during training.

Dropout trains several variations of a fixed-sized architecture, with each variation having different randomized nodes left out of the architecture.

Neurons cannot rely on each other since they may be deactivated during training. This encourages each neuron to learn more meaningful and generalizable features.

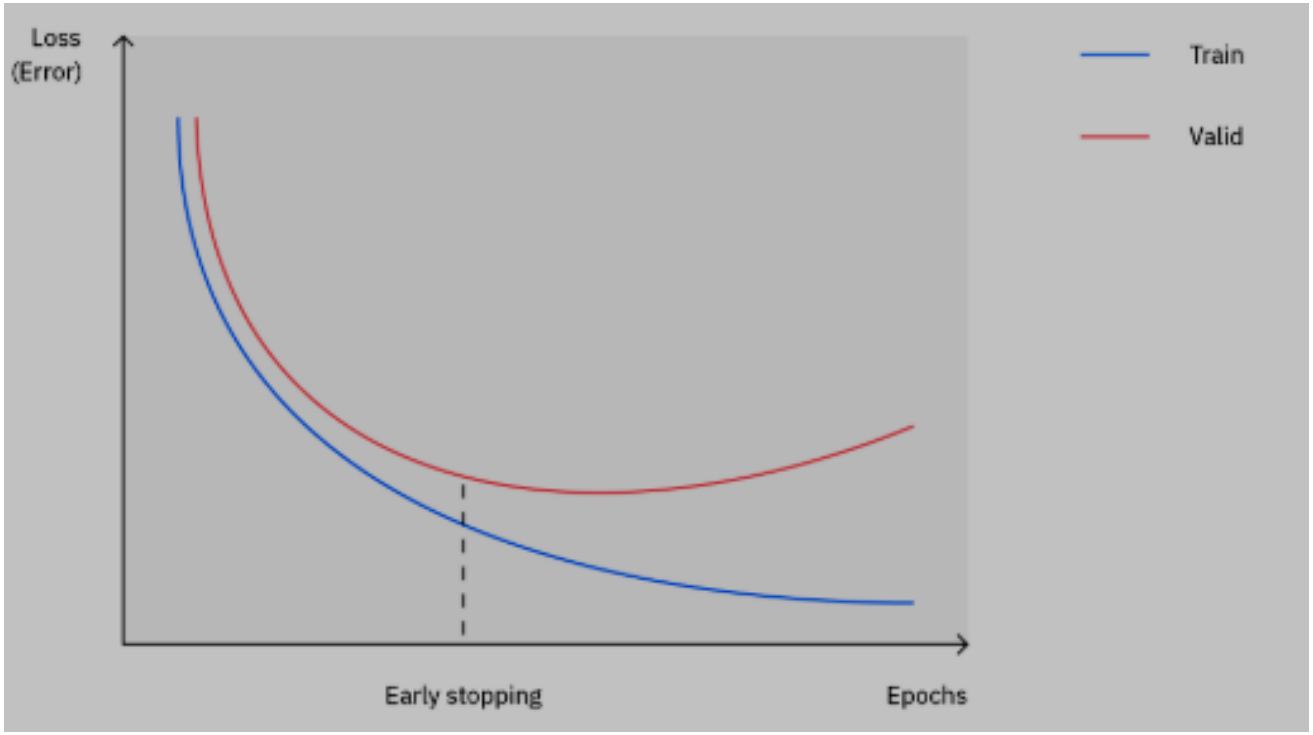


3. Batch Normalization

- Batch Normalization (BatchNorm) is a technique used to stabilize and speed up the training of neural networks. It normalizes the inputs of each layer by maintaining their mean and variance at consistent levels during training. This helps the network converge faster and generalize better.
- Normalizes the inputs of each layer to have zero mean and unit variance.
- Stabilizes and accelerates training while providing slight regularization.

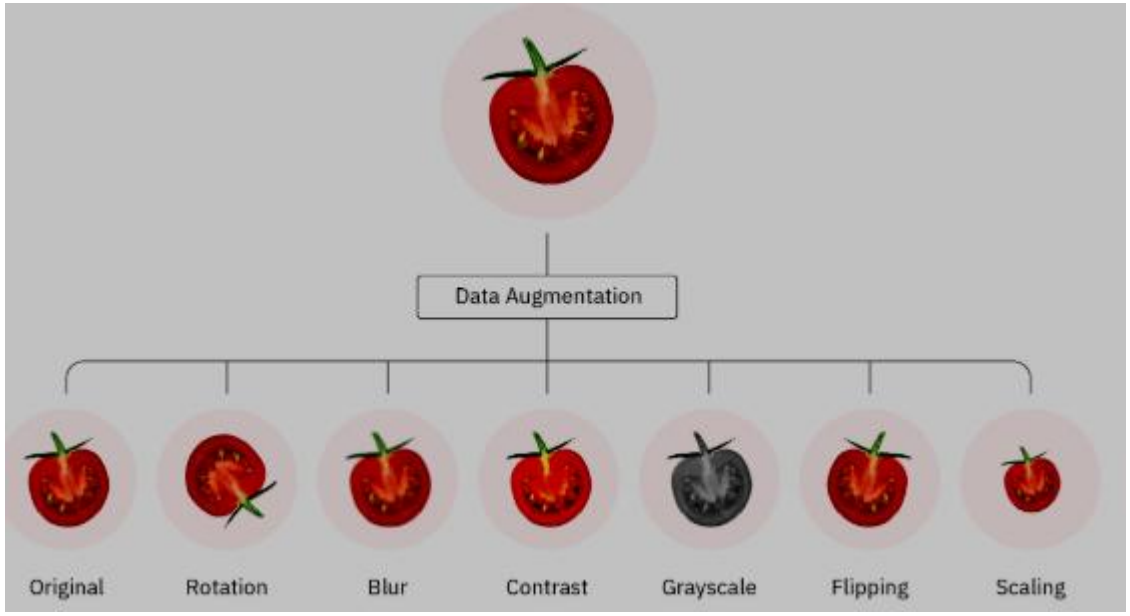
4. Early Stopping

Stops training when the model's performance on the validation set starts degrading, indicating overfitting.



5. Data Augmentation

Increases the training dataset size by creating modified versions of existing samples (e.g., rotating, flipping, scaling images).



6. Learning Rate Scheduling

- Learning rate scheduling is a strategy to adjust the learning rate during training to optimize model performance. Instead of using a fixed learning rate, the scheduler dynamically decreases or increases the learning rate at specified intervals or conditions.
- This helps the model converge faster, avoid local minima, and achieve better generalization.

Difficulty of training deep neural networks

Training deep neural networks can be challenging due to several factors:

1. Vanishing and Exploding Gradients: During backpropagation, the gradients of loss with respect to weights in early layers may become extremely small (vanishing) or extremely large (exploding).

- Vanishing gradients make it difficult for weights in earlier layers to update, slowing or stopping training.
- Exploding gradients lead to unstable weight updates.

Solutions:

- Use activation functions like ReLU, Leaky ReLU, or GELU.
- Apply weight initialization techniques like Xavier or He initialization.
- Use gradient clipping to address exploding gradients.

2. Computational Complexity: Deep networks require substantial computational resources, including GPUs, for faster and efficient training.

- Deep networks often require high computational resources due to the large number of parameters and complex matrix operations.

Solutions:

- Utilize GPUs and TPUs for parallel computation.
- Optimize using libraries like TensorFlow and PyTorch.

3. Overfitting: Deep models often have too many parameters, leading them to memorize training data rather than generalizing to unseen data.

Deep networks tend to memorize the training data rather than generalizing it.

Solutions:

- Use regularization techniques like L1, L2, or dropout.
- Employ data augmentation to artificially increase training data.
- Apply early stopping

4. **Optimization Challenges:** The loss surface of deep networks is highly complex, with numerous local minima and saddle points. This complexity makes gradient-based optimization challenging.

Gradient Descent Optimization:

- The loss surface of deep networks is often non-convex and contains multiple local minima, saddle points, and plateaus.
- Optimization algorithms like Adam, RMSProp, and learning rate schedules help alleviate these issues.

Solutions:

- Use advanced optimizers like Adam, RMSProp, or learning rate schedules.
- Employ batch normalization to stabilize training.

5. Choosing Hyperparameters: Choosing the right number of layers, learning rate, activation functions, and regularization parameters requires trial and error.

Solutions:

- Use hyperparameter optimization methods like grid search, random search, or Bayesian optimization.

6. Training Time: Due to their complexity, deep networks may take hours, days, or even weeks to converge to a reasonable solution.

Solutions:

- Use mini-batch gradient descent for faster updates.
- Distributed training across multiple GPUs.

Greedy Layerwise Training (Pretraining)

To address following difficulties in training deep neural networks, **greedy layerwise pretraining** was introduced as a strategy:

- **Better Initialization:** Avoids poor random initialization and helps networks start closer to a good solution.
- **Reduced Vanishing Gradient Issue:** Since each layer is trained individually, the gradients for early layers are preserved.
- **Faster Convergence:** Greedy pretraining often accelerates the overall convergence process.

Greedy Layerwise Training (Pretraining)

- A method for layer-by-layer training deep neural networks is called greedy layer-wise pre-training.
- It includes training each layer of a neural network individually in a greedy way, starting with the first layer and advancing to the last layer.
- The idea behind this approach is that it allows each layer to learn features that are useful for the next layer, thereby improving the overall performance of the network.

How It Works

1. Train each layer sequentially:

- Start by training the first layer using input features.
- Freeze the weights of the first layer and train the second layer using the outputs of the first layer.
- Repeat this process until all layers are trained.

2. Fine-tune the entire network: After all layers have been pretrained, fine-tune the entire network using backpropagation.

Numerical

- Train Similar network with greedy layerwise training

Step 1: Train First Hidden Layer (Autoencoder Approach)

Forward Pass for Layer 1

- Input: $X = [0.5, 0.2, 0.8]$
- Random Weights for first hidden layer:

$$W_1 = \begin{bmatrix} 0.3 & 0.6 & 0.1 \\ 0.4 & 0.2 & 0.5 \end{bmatrix}$$

- Compute activations:

$$Z_1 = W_1 X = \begin{bmatrix} 0.3 & 0.6 & 0.1 \\ 0.4 & 0.2 & 0.5 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.2 \\ 0.8 \end{bmatrix}$$

$$Z_1 = \begin{bmatrix} (0.3)(0.5) + (0.6)(0.2) + (0.1)(0.8) \\ (0.4)(0.5) + (0.2)(0.2) + (0.5)(0.8) \end{bmatrix} = \begin{bmatrix} 0.15 + 0.12 + 0.08 \\ 0.2 + 0.04 + 0.4 \end{bmatrix} = \begin{bmatrix} 0.35 \\ 0.64 \end{bmatrix}$$

- Apply sigmoid activation:

$$H_1 = \sigma(Z_1) = \frac{1}{1 + e^{-Z_1}}$$

$$H_1 = \begin{bmatrix} \frac{1}{1+e^{-0.35}} \\ \frac{1}{1+e^{-0.64}} \end{bmatrix} = \begin{bmatrix} 0.5866 \\ 0.654 \end{bmatrix}$$

This output H_1 will now be treated as input to the next layer.

- Train only this layer first using autoencoder loss (e.g., reconstruction error).

Step 2: Freeze Layer 1, Train Layer 2

- Input to Layer 2: $H_1 = [0.5866, 0.654]$
- Random Weights for second hidden layer:

$$W_2 = \begin{bmatrix} 0.5 & 0.3 \\ 0.7 & 0.2 \end{bmatrix}$$

- Compute:

$$Z_2 = W_2 H_1 = \begin{bmatrix} 0.5 & 0.3 \\ 0.7 & 0.2 \end{bmatrix} \begin{bmatrix} 0.5866 \\ 0.654 \end{bmatrix}$$

$$Z_2 = \begin{bmatrix} (0.5)(0.5866) + (0.3)(0.654) \\ (0.7)(0.5866) + (0.2)(0.654) \end{bmatrix} = \begin{bmatrix} 0.2933 + 0.1962 \\ 0.4106 + 0.1308 \end{bmatrix} = \begin{bmatrix} 0.4895 \\ 0.5414 \end{bmatrix}$$

- Apply **sigmoid**:

$$H_2 = \begin{bmatrix} \sigma(0.4895) \\ \sigma(0.5414) \end{bmatrix} = \begin{bmatrix} 0.6201 \\ 0.6321 \end{bmatrix}$$

- Train only this layer using **supervised learning.**

Step 3: Freeze Layer 2, Train Layer 3

- Repeat similar steps for Layer 3.

Final Step: Train Output Layer

- The final layer takes H_3 and maps it to the output using a **single neuron with a sigmoid.**
- We update **only the last layer** during supervised training.

Greedy Layerwise Training	Standard Backpropagation
Train one layer at a time, freezing previous layers	Train all layers together using backpropagation
Each layer is pre-trained separately (e.g., autoencoder for unsupervised learning)	All layers initialized randomly and optimized together
Slower initially but provides better initialization	Faster training but may suffer from vanishing gradients
Helps deep networks converge faster	Deep networks require careful tuning (e.g., batch normalization)